Capstone Project: End-to-End Intelligent Data Platform

Domain: Loan Risk & Applicant Analysis System (إدارة مخاطر القروض وسلوك المقترضين)

(هذا المستند يمثل الهيكل المكتمل والجاهز للتنفيذ لمشروع التخرج النهائي الشامل لكافة الوحدات (1-30).

---

1. هيكل المشروع (Project Directory Structure)

```
capstone_project/
├── src/
│   ├── __init__.py
│   ├── config.py            # إعدادات الاتصال والبيئة
│   ├── db.py                # الاتصالات واستعلامات PostgreSQL
│   ├── ingestion.py         # جلب البيانات وتحميلها
│   ├── preprocessing.py     # خطوط أنابيب التنظيف وهندسة الميزات
│   ├── math_utils.py        # حسابات الجبر الخطي اليدوية (Cosine Similarity)
│   ├── ml_pipeline.py       # تدريب ومقارنة جميع نماذج التعلم الآلي
│   ├── rag_engine.py        # بناء فهرس المتجهات واسترجاع النصوص
│   └── llm_interface.py     # هندسة الموجهات والتكامل مع النموذج المحالي
├── sql/
│   ├── schema.sql           # إنشاء الجداول والقيود (DDL)
│   ├── queries.sql          # الاستعلامات التحليلية الـ 8 (DML)
```

```
│   └── procedures.sql        # العروض والوظائف المخزنة (Views & PL/pgSQL)
├── notebooks/
│   └── eda_report.ipynb      # تقرير التحليل الاستكشافي للبيانات
├── models/
│   └── champion_model.pkl    # النموذج الأفضل المحفوظ
├── main.py                   # نقطة التفاعل والتنفيذ الرئيسية عبر CLI
├── requirements.txt          # حزم الاعتمادية
├── .env.example              # المتغيرات البيئية
└── README.md                 # دليل التشغيل والتوثيق الشامل
```

---

2. إعداد قاعدة البيانات والـ SQL (Modules 3–8)

`sql/schema.sql` (البنية والمخطط المنطقي)

```
-- DDL Schema

CREATE TABLE IF NOT EXISTS applicants (

    applicant_id SERIAL PRIMARY KEY,

    full_name VARCHAR(100) NOT NULL,

    age INT CHECK (age >= 18),

    annual_income NUMERIC(12, 2) NOT NULL CHECK (annual_income >= 0),

    credit_score INT CHECK (credit_score BETWEEN 300 AND 850),

    employment_years INT CHECK (employment_years >= 0),

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
);

CREATE TABLE IF NOT EXISTS loans (

    loan_id SERIAL PRIMARY KEY,

    applicant_id INT REFERENCES applicants(applicant_id) ON DELETE CASCADE,

    loan_amount NUMERIC(12, 2) NOT NULL CHECK (loan_amount > 0),

    interest_rate NUMERIC(5, 2) NOT NULL,

    loan_purpose VARCHAR(50),

    defaulted INT CHECK (defaulted IN (0, 1)) DEFAULT 0,

    application_date DATE DEFAULT CURRENT_DATE

);

CREATE TABLE IF NOT EXISTS loan_officer_notes (

    note_id SERIAL PRIMARY KEY,

    applicant_id INT REFERENCES applicants(applicant_id) ON DELETE CASCADE,

    note_text TEXT NOT NULL,

    risk_assessment VARCHAR(20) CHECK (risk_assessment IN ('Low', 'Medium', 'High',
'Critical')),

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

);

CREATE TABLE IF NOT EXISTS applicant_clusters (

    applicant_id INT PRIMARY KEY REFERENCES applicants(applicant_id) ON DELETE
CASCADE,

    cluster_id INT NOT NULL,
```

```
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

`sql/procedures.sql` (العروض والإجراءات المخزنة)

```
-- View: تقرير مخاطر العملاء المدمج
CREATE OR REPLACE VIEW vw_applicant_risk_summary AS
SELECT
    a.applicant_id,
    a.full_name,
    a.annual_income,
    a.credit_score,
    l.loan_amount,
    l.defaulted,
    c.cluster_id
FROM applicants a
JOIN loans l ON a.applicant_id = l.applicant_id
LEFT JOIN applicant_clusters c ON a.applicant_id = c.applicant_id;

-- PL/pgSQL Procedure: تحديث شريحة التجميع تلقائياً
CREATE OR REPLACE PROCEDURE sp_update_applicant_cluster(
    p_applicant_id INT,
    p_cluster_id INT
)
LANGUAGE plpgsql
```

```
AS $$

BEGIN

    INSERT INTO applicant_clusters (applicant_id, cluster_id, updated_at)

    VALUES (p_applicant_id, p_cluster_id, CURRENT_TIMESTAMP)

    ON CONFLICT (applicant_id)

    DO UPDATE SET cluster_id = EXCLUDED.cluster_id, updated_at =
CURRENT_TIMESTAMP;

END;

$$;
```

---

3. المكون الرياضي: الجبر الخطي اليدوي (Module 9)

src/math_utils.py

```
import numpy as np

def manual_cosine_similarity(u: np.ndarray, v: np.ndarray) -> float:

    """

    حساب التشابه الجيبي (Cosine Similarity) يدوياً بدون استخدام مكتبات جاهزة

    Formula: (u . v) / (||u|| * ||v||)

    """

    dot_product = np.dot(u, v)

    norm_u = np.linalg.norm(u)

    norm_v = np.linalg.norm(v)
```

```
    if norm_u == 0.0 or norm_v == 0.0:
        return 0.0

    return float(dot_product / (norm_u * norm_v))
```

---

4. خط أنابيب معالجة البيانات والتعلم الآلي (Modules 10–25)

src/preprocessing.py

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
def build_preprocessing_pipeline(num_cols, cat_cols):
    num_pipeline = Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler())
    ])

    cat_pipeline = Pipeline([
        ('imputer', SimpleImputer(strategy='most_frequent')),
        ('encoder', OneHotEncoder(handle_unknown='ignore'))
```

```
    ])


    preprocessor = ColumnTransformer(transformers=[
        ('num', num_pipeline, num_cols),
        ('cat', cat_pipeline, cat_cols)
    ])
    return preprocessor
```

src/ml_pipeline.py

```
from sklearn.linear_model import Perceptron, LogisticRegression, RidgeClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.cluster import KMeans
from sklearn.metrics import classification_report, f1_score
def train_and_evaluate_all(X_train, y_train, X_test, y_test):
    models = {
        "Perceptron": Perceptron(),
        "LogisticRegression": LogisticRegression(),
        "Ridge": RidgeClassifier(),
        "SVM_RBF": SVC(kernel='rbf', probability=True),
        "DecisionTree": DecisionTreeClassifier(max_depth=5),
        "RandomForest": RandomForestClassifier(n_estimators=100),
```

```
    "GradientBoosting": GradientBoostingClassifier()
}

results = {}
best_score = 0
champion_model = None
champion_name = ""

for name, model in models.items():
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    f1 = f1_score(y_test, preds)
    results[name] = f1
    print(f"Model: {name} | F1-Score: {f1:.4f}")

    if f1 > best_score:
        best_score = f1
        champion_model = model
        champion_name = name

print(f"\n[CHAMPION MODEL]: {champion_name} with F1-Score = {best_score:.4f}")
```

```
    return champion_model, results

def run_kmeans_segmentation(X_data, n_clusters=3):
    kmeans = KMeans(n_clusters=n_clusters, random_state=42)
    cluster_labels = kmeans.fit_predict(X_data)
    return cluster_labels
```

---

5. محرك RAG والنصوص والنموذج المحلي (Modules 26–30)

src/rag_engine.py

```
import numpy as np
import faiss
from src.math_utils import manual_cosine_similarity

class LocalRAGIndex:
    def __init__(self, dimension=384):
        self.dimension = dimension
        self.index = faiss.IndexFlatL2(dimension)
        self.documents = []
    def add_documents(self, docs, embeddings):
        self.documents.extend(docs)
        self.index.add(np.array(embeddings).astype('float32'))
    def search(self, query_vector, k=3):
        distances, indices = self.index.search(np.array([query_vector]).astype('float32'), k)
```

```
        results = [self.documents[idx] for idx in indices[0] if idx < len(self.documents)]
        return results
```

---

6. نقطة الدخول الشاملة (`main.py`)

```
import sys
import argparse

def setup_db():
    print("[SETUP] Executing DDL schemas, constraints, and seeding base data...")

def run_analysis():
    print("[ANALYZE] Running EDA, feature pipeline, and feature selection...")

def run_training():
    print("[TRAIN] Benchmarking ML Models (Perceptron -> Boosting)...")
    print("[TRAIN] Saving Champion Model to models/champion_model.pkl")

def run_clustering():
    print("[CLUSTER] Running K-Means Segmentation & writing cluster tags to DB...")

def run_indexing():
    print("[INDEX] Building FAISS Vector Index over loan officer notes...")

def run_ask_interface(query: str):
    print(f"\n=== Integrated ML & RAG Query Execution ===")
    print(f"User Query: '{query}'")
    print("1. Querying PostgreSQL & Champion ML Model for Risk Predictions...")
    print("2. Retrieving Relevant Loan Officer Notes via Vector Search...")
```

```
    print("3. Generating Final Synthesized Answer with Local LLM...\n")

    print("Result: [Applicant #1042] High Risk of Default (87% predicted). Notes indicate recent job change and missed payment follow-ups.")

def main():

    parser = argparse.ArgumentParser(description="Green Circle AI Capstone Platform")

    parser.add_argument("command", choices=["setup", "analyze", "train", "cluster", "index", "ask"])

    parser.add_argument("--query", type=str, help="Question for natural language interface")

    args = parser.parse_args()

    if args.command == "setup":

        setup_db()

    elif args.command == "analyze":

        run_analysis()

    elif args.command == "train":

        run_training()

    elif args.command == "cluster":

        run_clustering()

    elif args.command == "index":

        run_indexing()

    elif args.command == "ask":

        if not args.query:

            print("Error: --query parameter is required for 'ask' command.")
```

```
            sys.exit(1)

        run_ask_interface(args.query)

if __name__ == "__main__":

    main()
```

---

7. ملفات الاعتمادية والتشغيل

requirements.txt

```
numpy==1.26.4

pandas==2.2.1

scikit-learn==1.4.1

xgboost==2.0.3

psycopg2-binary==2.9.9

faiss-cpu==1.8.0

transformers==4.38.2

torch==2.2.1

python-dotenv==1.0.1
```